

ShareGear — Database Design v2.0

Principal Solution Architect · Senior Database Architect · Lead .NET Technical Reviewer

Document Version: 2.0 — Customer Service, Field Operations & Evidence Forms Stack: EF Core · SQL Server · ASP.NET Core (.NET 8) Supersedes:
ShareGear_MVP_Database_Design.md (v1.0)

Change Log — v1.0 → v2.0

#	Change Type	Description
1	DROP	<code>Complaints</code> table removed entirely
2	ADD	<code>Tickets</code> — full customer service ticket entity (replaces Complaints)
3	ADD	<code>TicketMessages</code> — threaded follow-up conversation per ticket
4	ADD	<code>TicketAttachments</code> — per-message file & photo evidence uploads
5	ADD	<code>EmployeeProfiles</code> — ShareGear field/CS employee profile
6	ADD	<code>JobCompletionEvidenceForms</code> — provider photo-proof form on job completion
7	ADD	<code>JobCompletionEvidencePhotos</code> — provider-uploaded completion photos
8	ADD	<code>FieldVerificationForms</code> — employee field visit dispatch & outcome record
9	ADD	<code>FieldVerificationPhotos</code> — employee on-site photos for AI comparison
10	MODIFY	<code>Users.Role</code> enum gains <code>Employee = 3</code>
11	MODIFY	<code>Bookings</code> — two new status values + <code>FieldVerificationDispatchedAt</code> timestamp
12	MODIFY	Booking state machine — adds <code>PENDING_FIELD_VERIFICATION</code> path
13	MODIFY	<code>PlatformSettings</code> — two new seeded keys for field ops SLAs

Table count: 26 (v1) – 1 (Complaints) + 8 (new) = 33 tables Justified exception to the 18–28 target: the full ticket system and two separate evidence form entities each have distinct

schemas, distinct relationships, and distinct business workflows. Merging any of them would destroy query clarity and form-level validation boundaries.

1. New & Updated Business Domains

The following domains are added to the v1 domain table. All other v1 domains are unchanged.

Domain	Responsibility
Customer Service — Tickets	Threaded ticket lifecycle, agent assignment, follow-up messages, internal notes, resolution tracking
Field Operations	Employee dispatch, on-site photo capture, AI similarity scoring, admin decision on frozen escrow
Job Completion Evidence	Provider-submitted photo proof form on every booking completion — primary reference for field comparison

2. Updated User Roles

The `UserRole` enum gains a fourth value. All existing role definitions from v1 are unchanged.

```
csharp
public enum UserRole
{
    Customer = 0,
    Provider = 1,
    Admin    = 2,
    Employee = 3 // ShareGear internal - CS agents and field verification sta
}
```

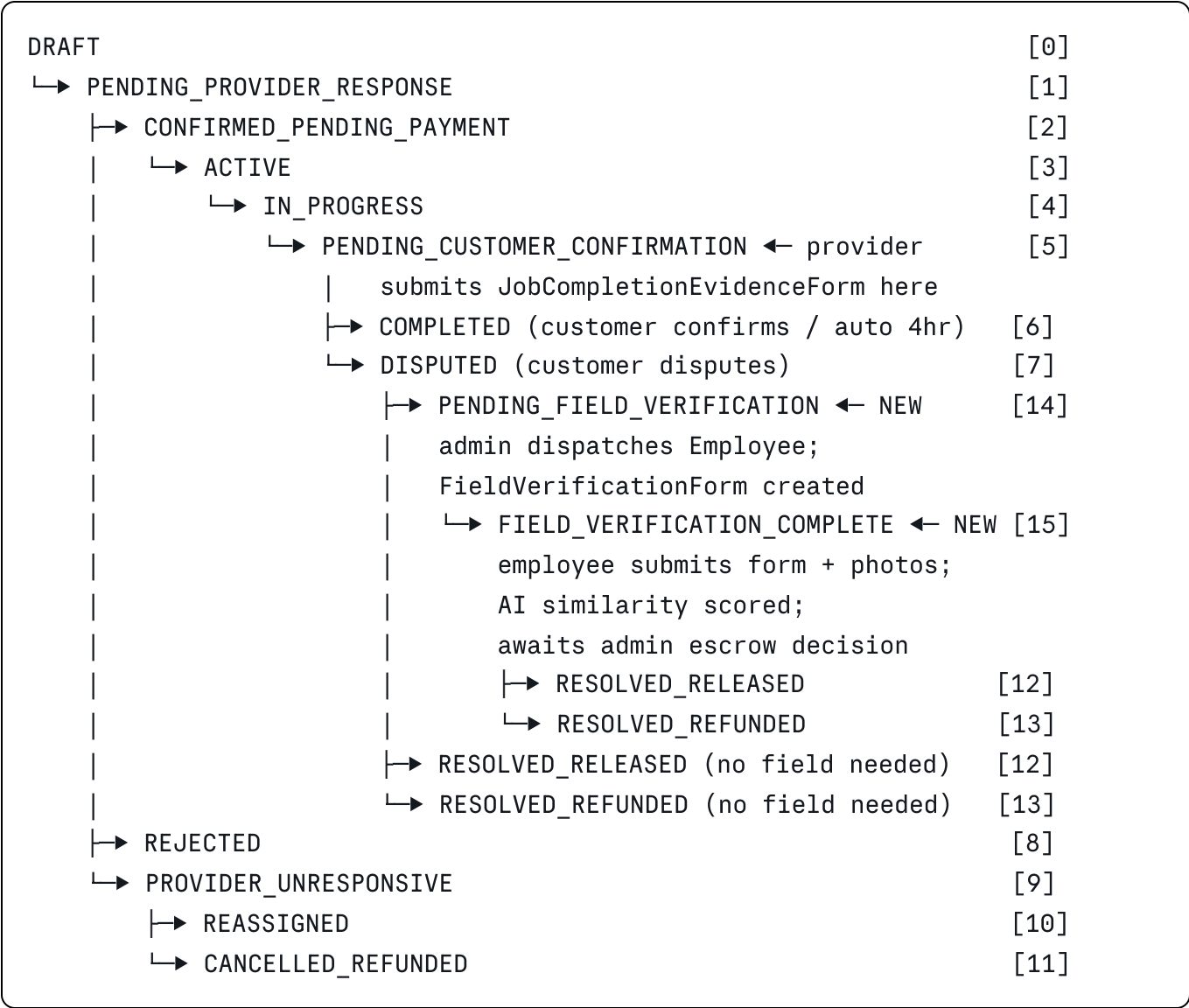
Employee sub-roles are not a separate enum. The `EmployeeProfiles.Department` string column and the `EmployeeProfiles.IsAvailableForDispatch` flag together define what an employee can do. A CS-only agent has `IsAvailableForDispatch = false`. A field operative has `IsAvailableForDispatch = true`. Both share the same `Role = Employee` claim, and

RBAC policies at the API level distinguish ticket-management endpoints from field-form endpoints.

3. Updated State Machines

3.1 Booking Status — Extended

Two new statuses are added. All v1 statuses are preserved with their original integer values.



Updated `BookingStatus` enum:

```
csharp
```

}

3.2 JobCompletionEvidenceForm Status

[illegible]

csharp

```
public enum EvidenceFormStatus
{
    Draft = 0,
    Submitted = 1,
    UnderAdminReview = 2,
    AcceptedAsEvidence = 3,
    RejectedAsEvidence = 4
}
```

3.3 FieldVerificationForm Status

```
DISPATCHED → ON_SITE → COMPLETED → [Admin reviews AiSimilarityScore]
      ↳ FAILED ACCESS                → RESOLVED RELEASED
```

→ RESOLVED_REFUNDED
→ PARTIAL_SETTLEMENT

csharp

```
public enum VisitStatus
{
    Dispatched      = 0,
    OnSite          = 1,
    Completed       = 2,
    FailedAccess    = 3
}

public enum FieldVerificationOutcome
{
    JobConfirmed      = 0,
    JobNotConfirmed   = 1,
    PartiallyConfirmed = 2,
    Inconclusive      = 3
}

public enum FieldVerificationAdminDecision
{
    Pending           = 0,
    ReleaseToProvider = 1,
    RefundToCustomer  = 2,
    PartialSettlement = 3
}
```

3.4 Ticket Status

csharp

```

public enum TicketStatus
{
    Open                = 0,
    InProgress          = 1,
    PendingCustomerReply = 2,
    PendingProviderReply = 3,
    PendingFieldVerification = 4, // mirrors booking state; ticket locked until
    Resolved            = 5,
    Closed              = 6,
    Reopened            = 7
}

public enum TicketCategory
{
    ServiceQuality      = 0,
    OperatorConduct     = 1,
    PaymentIssue        = 2,
    Fraud               = 3,
    SafetyConcern       = 4,
    CompletionDispute   = 5, // auto-created when booking enters DISPUTED
    MarketplaceIssue    = 6,
    Other               = 7
}

public enum TicketResolutionType
{
    FullRefundToCustomer = 0,
    FullReleaseToProvider = 1,
    PartialSettlement    = 2,
    ServiceRedo          = 3,
    NoActionRequired     = 4,
    EscalatedToFieldVerification = 5
}

```

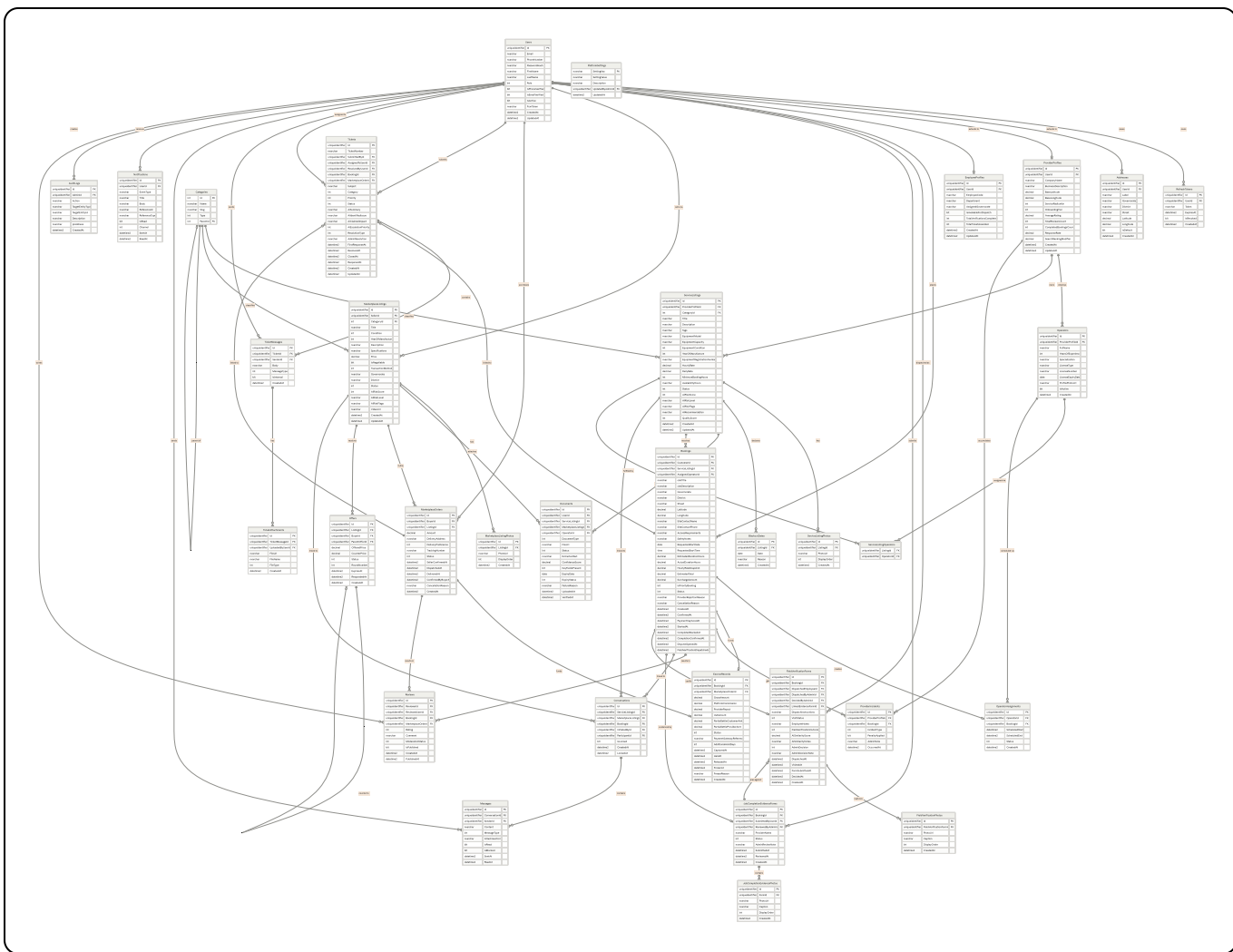
4. Updated Entity List — 33 Tables

Rows marked **NEW** are added in v2. Complaints (v1 #20) is replaced. All other v1 tables retain their original numbering and are unchanged unless noted.

#	Table	Status	Purpose
1	<code>Users</code>	Modified	Core identity — Role enum gains Employee=3
2	<code>RefreshTokens</code>	Unchanged	JWT refresh token store
3	<code>Addresses</code>	Unchanged	User saved addresses
4	<code>ProviderProfiles</code>	Unchanged	Provider geo-zone, tier, cached metrics
5	<code>EmployeeProfiles</code>	NEW	ShareGear CS agent and field operative profile
6	<code>Operators</code>	Unchanged	Equipment operators under a provider
7	<code>Categories</code>	Unchanged	Hierarchical listing taxonomy
8	<code>ServiceListings</code>	Unchanged	Service listing with AI moderation state
9	<code>ServiceListingPhotos</code>	Unchanged	Photo gallery for service listings
10	<code>ServiceListingOperators</code>	Unchanged	Junction: operators per listing
11	<code>BlackoutDates</code>	Unchanged	Provider-declared unavailability
12	<code>Documents</code>	Unchanged	Verification documents + OCR results
13	<code>Bookings</code>	Modified	+ 2 new statuses + <code>FieldVerificationDispatchedAt</code>
14	<code>OperatorAssignments</code>	Unchanged	Operator scheduling blocks
15	<code>EscrowRecords</code>	Unchanged	Unified escrow for bookings + orders
16	<code>MarketplaceListings</code>	Unchanged	Used equipment / spare parts listings
17	<code>MarketplaceListingPhotos</code>	Unchanged	Photo gallery for marketplace listings
18	<code>MarketplaceOrders</code>	Unchanged	Marketplace purchase lifecycle
19	<code>Offers</code>	Unchanged	Negotiation rounds on negotiable listings
20	<code>Reviews</code>	Unchanged	Post-transaction reviews
21	<code>Complaints</code> → <code>Tickets</code>	REPLACED	Full customer service ticket

#	Table	Status	Purpose
22	TicketMessages	NEW	Threaded follow-up per ticket
23	TicketAttachments	NEW	Per-message file and photo attachments
24	JobCompletionEvidenceForms	NEW	Provider photo-proof form on completion
25	JobCompletionEvidencePhotos	NEW	Provider-uploaded completion photos
26	FieldVerificationForms	NEW	Employee dispatch record and site visit outcome
27	FieldVerificationPhotos	NEW	Employee on-site photos for AI comparison
28	Conversations	Unchanged	Chat thread header
29	Messages	Unchanged	Individual chat messages
30	Notifications	Unchanged	User notification inbox
31	ProviderIncidents	Unchanged	Incident log; drives penalty rules
32	AuditLogs	Unchanged	Immutable admin action log
33	PlatformSettings	Modified	+ 2 new field-ops SLA keys

5. Complete ERD — v2.0 (Mermaid)



6. New & Modified Table Definitions

The eight new tables and the two modified tables are fully specified below. All 25 unchanged tables retain their v1.0 definitions exactly as documented.

6.1 **EmployeeProfiles** — NEW

Purpose: Extends **Users** for the **Employee** role (Role = 3). Separates employee operational metadata from the core identity record, consistent with how **ProviderProfiles** handles the provider role extension. A CS-only agent and a field operative are both employees; **IsAvailableForDispatch** is the capability flag that determines which forms they can be assigned to.

Column	Type	Nullable	Notes
<code>Id</code>	<code>uniqueidentifier</code>	NO	PK
<code>UserId</code>	<code>uniqueidentifier</code>	NO	FK → Users; unique constraint — one profile per employee
<code>EmployeeCode</code>	<code>nvarchar(20)</code>	NO	Unique; human-readable (e.g., <code>EMP-00042</code>); assigned at onboarding
<code>Department</code>	<code>nvarchar(100)</code>	YES	Free text: "Field Operations", "Customer Service", "Operations"
<code>AssignedGovernorate</code>	<code>nvarchar(100)</code>	YES	Primary governorate for field dispatch routing
<code>IsAvailableForDispatch</code>	<code>bit</code>	NO	Default <code>false</code> ; <code>true</code> = eligible for <code>FieldVerificationForm</code> assignments
<code>TotalVerificationsCompleted</code>	<code>int</code>	NO	Default 0; cached count updated after each FV form submission
<code>TotalTicketsHandled</code>	<code>int</code>	NO	Default 0; cached count updated when ticket is resolved
<code>CreatedAt</code>	<code>datetime2</code>	NO	UTC
<code>UpdatedAt</code>	<code>datetime2</code>	NO	UTC

EF Core mapping:

```
csharp
```

```
entity.HasIndex(e => e.UserId).IsUnique();  
entity.HasIndex(e => e.EmployeeCode).IsUnique();  
entity.HasOne(e => e.User).WithOne(u => u.EmployeeProfile)  
    .HasForeignKey<EmployeeProfile>(e => e.UserId);
```

6.2 **Tickets** — REPLACES **Complaints**

Purpose: Full customer service ticket. Covers general enquiries, payment disputes, quality complaints, and — critically — **CompletionDispute** tickets that are auto-created when a booking enters **DISPUTED** status. The ticket is the communication and tracking layer; escrow and booking state changes are driven separately through their own state machines. The ticket surfaces all of that context to the assigned employee.

Column	Type	Nullable	Notes
Id	uniqueidentifier	NO	PK
TicketNumber	nvarchar(20)	NO	Unique; human-readable (e.g., TKT-20240001); generated by service layer
SubmittedById	uniqueidentifier	NO	FK → Users; customer or provider who opened the ticket
AssignedToUserId	uniqueidentifier	YES	FK → Users; Employee or Admin assigned to handle
ResolvedByUserId	uniqueidentifier	YES	FK → Users; who closed the ticket
BookingId	uniqueidentifier	YES	FK → Bookings; set for service-related tickets
MarketplaceOrderId	uniqueidentifier	YES	FK → MarketplaceOrders; set for marketplace-related tickets
Subject	nvarchar(200)	NO	Brief subject line
Category	int	NO	Enum: TicketCategory
Priority	int	NO	Default 0 (Normal); set by AI on first message processing
Status	int	NO	Default 0 (Open); TicketStatus enum
AiSummary	nvarchar(max)	YES	ModeratorAgent neutral summary of the first message
AiIdentifiedIssue	nvarchar(500)	YES	Core issue extracted by AI
AiClaimedImpact	nvarchar(500)	YES	Claimed impact extracted by AI
AiEscalationPriority	int	YES	0=Normal, 1=High, 2=Urgent — AI-computed
ResolutionType	int	YES	TicketResolutionType enum; set at resolution

Column	Type	Nullable	Notes
AdminResolution	nvarchar(max)	YES	Admin-authored resolution note sent to submitter
FirstResponseAt	datetime2	YES	Timestamp of first agent <code>TicketMessage</code> ; SLA measurement anchor
ResolvedAt	datetime2	YES	UTC
ClosedAt	datetime2	YES	UTC; after customer acknowledges resolution
ReopenedAt	datetime2	YES	UTC; set if customer reopens a resolved ticket
CreatedAt	datetime2	NO	UTC
UpdatedAt	datetime2	NO	UTC; updated on any status change

Business rules (enforced in service layer):

- A `CompletionDispute` ticket is auto-created by `BookingService` whenever `Bookings.Status` transitions to `Disputed`. `BookingId` is set, `SubmittedById` = the customer, `Priority` = High default before AI processes.
- Only one active (non-Closed) ticket per `BookingId`. Enforced by a filtered unique index: `CREATE UNIQUE INDEX UX_Tickets_BookingId_Active ON Tickets (BookingId) WHERE BookingId IS NOT NULL AND Status NOT IN (6)` — Status 6 = Closed.
- `CHECK` constraint: `BookingId IS NULL OR MarketplaceOrderId IS NULL` — a ticket links to at most one transaction context.

EF Core mapping:

```
csharp
entity.HasIndex(e => e.TicketNumber).IsUnique();
entity.HasOne(e => e.SubmittedBy).WithMany().HasForeignKey(e => e.SubmittedById
entity.HasOne(e => e.AssignedTo).WithMany().HasForeignKey(e => e.AssignedToUser
    .IsRequired(false);
entity.HasOne(e => e.ResolvedBy).WithMany().HasForeignKey(e => e.ResolvedByUser
    .IsRequired(false);
```

6.3 TicketMessages — NEW

Purpose: Threaded conversation thread within a ticket. Every communication between the customer, provider, assigned agent, and system is a message. This replaces the single-field `ProviderReply` and `AdminResolution` text approach of the old Complaints table with a full audit trail of the conversation history.

Column	Type	Nullable	Notes
<code>Id</code>	<code>uniqueidentifier</code>	NO	PK
<code>TicketId</code>	<code>uniqueidentifier</code>	NO	FK → Tickets
<code>SenderId</code>	<code>uniqueidentifier</code>	NO	FK → Users; author of the message
<code>Body</code>	<code>nvarchar(max)</code>	NO	Message content; max 3,000 characters enforced at application layer
<code>MessageType</code>	<code>int</code>	NO	Enum: <code>CustomerMessage=0</code> , <code>ProviderMessage=1</code> , <code>AgentMessage=2</code> , <code>SystemMessage=3</code>
<code>IsInternal</code>	<code>bit</code>	NO	Default <code>false</code> ; <code>true</code> = internal agent note, invisible to customer and provider
<code>CreatedAt</code>	<code>datetime2</code>	NO	UTC

Design note on `IsInternal`: Internal notes allow CS agents to communicate with each other or leave decision context without the customer seeing it. This is a standard customer service pattern. The API endpoint that returns ticket messages for customer-facing views applies a `WHERE IsInternal = 0` filter. The admin/employee-facing view returns all messages.

`MessageType = SystemMessage` is written by the `TicketService` itself, not by a human sender. Examples: *"Ticket assigned to [Agent Name]"*, *"Booking entered DISPUTED status — escrow frozen"*, *"Field verification employee dispatched"*. `SenderId` for system messages points to a dedicated system user account (seeded at deployment).

6.4 TicketAttachments — NEW

Purpose: Files and photos attached to individual ticket messages. Normalised at the message level so that evidence submitted in a specific reply is contextually linked to that

point in the conversation. Querying all attachments for a ticket is a two-join operation ((TicketAttachments → TicketMessages → Tickets)) which is trivially cheap.

Column	Type	Nullable	Notes
Id	uniqueidentifier	NO	PK
TicketMessageId	uniqueidentifier	NO	FK → TicketMessages
UploadedByUserId	uniqueidentifier	NO	FK → Users
FileUrl	nvarchar(500)	NO	Azure Blob Storage URL
FileName	nvarchar(255)	NO	Original filename for display
FileType	int	NO	Enum: Image=0, Document=1, Video=2
CreatedAt	datetime2	NO	UTC

Limits enforced at application layer: Max 5 attachments per message. Max 10 MB per file for images; max 25 MB for documents. The ModeratorAgent's CheckImageContent() is invoked asynchronously on upload for image attachments.

6.5 JobCompletionEvidenceForms — NEW

Purpose: The provider-submitted photographic proof form that is created when an operator marks a booking as complete. This is the primary evidence record that proves the job was done. If the booking is subsequently disputed and a field verification employee is dispatched, the photos in this form become the comparison reference for the AI similarity assessment.

Trigger: BookingService.MarkJobComplete() — when the operator marks the job complete, the booking transitions to PENDING_CUSTOMER_CONFIRMATION and simultaneously the API requires the provider to submit this form (photos + notes) as part of the same UI action. The form is validated server-side before the status transition is committed.

Column	Type	Nullable	Notes
<code>Id</code>	<code>uniqueidentifier</code>	NO	PK
<code>BookingId</code>	<code>uniqueidentifier</code>	NO	FK → Bookings; unique — one form per booking
<code>SubmittedByUserId</code>	<code>uniqueidentifier</code>	NO	FK → Users; the provider account
<code>ReviewedByAdminId</code>	<code>uniqueidentifier</code>	YES	FK → Users; admin who reviewed the form during a dispute
<code>ProviderNotes</code>	<code>nvarchar(1000)</code>	YES	Provider's description of work performed
<code>Status</code>	<code>int</code>	NO	Default 1 (Submitted); <code>EvidenceFormStatus</code> enum
<code>AdminReviewNote</code>	<code>nvarchar(500)</code>	YES	Admin note on acceptance or rejection of evidence
<code>SubmittedAt</code>	<code>datetime2</code>	NO	UTC; timestamp of form submission
<code>ReviewedAt</code>	<code>datetime2</code>	YES	UTC; timestamp of admin review decision
<code>CreatedAt</code>	<code>datetime2</code>	NO	UTC

Business rules:

- Unique constraint on `BookingId` — `CREATE UNIQUE INDEX UX_JobCompletionEvidenceForms_BookingId ON JobCompletionEvidenceForms (BookingId)`.
- Minimum 1 photo required; maximum 10 photos. Enforced in `JobCompletionEvidenceService` before `Status` is set to `Submitted`.
- If no form is submitted within 30 minutes of `Bookings.CompletedMarkedAt`, a reminder notification is sent to the provider. The booking's 4-hour auto-confirm timer still runs regardless.
- Status transitions to `UnderAdminReview` when the linked booking enters `DISPUTED`. Admin and the assigned employee can see the form and its photos.

6.6 JobCompletionEvidencePhotos — NEW

Purpose: Individual photos attached to a JobCompletionEvidenceForm. These photos are taken by the operator at the job site to prove completion — e.g., photos of the completed task, the equipment on site, and the work area. They become the comparison baseline for FieldVerificationPhotos during AI similarity scoring.

Column	Type	Nullable	Notes
Id	uniqueidentifier	NO	PK
FormId	uniqueidentifier	NO	FK → JobCompletionEvidenceForms
PhotoUrl	nvarchar(500)	NO	Azure Blob Storage URL
Caption	nvarchar(200)	YES	Optional label from provider (e.g., "Completed wall section", "Equipment on roof")
DisplayOrder	int	NO	Default 0; controls gallery ordering
CreatedAt	datetime2	NO	UTC

Index: CREATE INDEX IX_JobCompletionEvidencePhotos_FormId ON JobCompletionEvidencePhotos (FormId, DisplayOrder) — supports ordered gallery retrieval.

6.7 FieldVerificationForms — NEW

Purpose: The central record for a ShareGear employee's dispatch to a job site. Created by an admin after deciding that a disputed booking requires physical field verification. The form tracks the full lifecycle: dispatch → on-site → photo submission → AI similarity scoring → admin decision → escrow resolution.

This form is the operational heart of the frozen-escrow resolution workflow. The admin cannot make an informed escrow decision without it. The AiSimilarityScore is computed by invoking the Azure OpenAI vision model via Semantic Kernel to compare the provider's JobCompletionEvidencePhotos with the employee's FieldVerificationPhotos, producing a 0.0-1.0 similarity confidence value.

Column	Type	Nullable	Notes
<code>Id</code>	<code>uniqueidentifier</code>	NO	PK
<code>BookingId</code>	<code>uniqueidentifier</code>	NO	FK → Bookings; unique — one FV form per booking
<code>DispatchedEmployeeId</code>	<code>uniqueidentifier</code>	NO	FK → Users; the Employee (Role=3) assigned to this visit
<code>DispatchedByAdminId</code>	<code>uniqueidentifier</code>	NO	FK → Users; the Admin who created the dispatch
<code>DecidedByAdminId</code>	<code>uniqueidentifier</code>	YES	FK → Users; Admin who made the final escrow decision
<code>LinkedEvidenceFormId</code>	<code>uniqueidentifier</code>	NO	FK → JobCompletionEvidenceForm the provider's photos to compare against
<code>DispatchInstructions</code>	<code>nvarchar(1000)</code>	YES	Admin-authored notes to the employee about site access, what to check
<code>VisitStatus</code>	<code>int</code>	NO	Default 0 (Dispatched); <code>VisitStatus</code> enum
<code>EmployeeNotes</code>	<code>nvarchar(2000)</code>	YES	Employee's on-site observations and findings
<code>FieldVerificationOutcome</code>	<code>int</code>	YES	<code>FieldVerificationOutcome</code> enum; set by employee on form submission
<code>AiSimilarityScore</code>	<code>decimal(5,4)</code>	YES	0.0000–1.0000; computed by Semantic Kernel after photo submission
<code>AiSimilarityNotes</code>	<code>nvarchar(1000)</code>	YES	AI narrative explaining the similarity assessment
<code>AdminDecision</code>	<code>int</code>	NO	Default 0 (Pending); <code>FieldVerificationAdminDecision</code> enum

Column	Type	Nullable	Notes
AdminDecisionNote	nvarchar(500)	YES	Admin's explanation of the escrow decision sent to both parties
DispatchedAt	datetime2	NO	UTC; when admin created the dispatch
VisitedAt	datetime2	YES	UTC; when employee updates status to OnSite
FormSubmittedAt	datetime2	YES	UTC; when employee submits completed form
DecidedAt	datetime2	YES	UTC; when admin makes the final escrow decision
CreatedAt	datetime2	NO	UTC

Business rules:

- Unique constraint on `BookingId` — only one field verification per booking.
- Creating this record transitions `Bookings.Status` to `PendingFieldVerification` and sets `Bookings.FieldVerificationDispatchedAt = GETUTCDATE()`.
- The employee is notified via the standard `Notifications` table (`EventType: FieldVerificationDispatched`) and receives the booking's `SiteContactName`, `SiteContactPhone`, and service address in the dispatch notification.
- `AiSimilarityScore` is populated asynchronously by a background service after `FormSubmittedAt` is set. The background service calls the Semantic Kernel agent with both photo sets. The booking status transitions to `FieldVerificationComplete` once the score is stored.
- `AdminDecision` can only be set when `VisitStatus = Completed` and `AiSimilarityScore IS NOT NULL`. The service layer enforces this guard.
- When `AdminDecision` is set to any value other than `Pending`, the `EscrowService` is invoked to execute the corresponding release/refund/split, and `Bookings.Status` transitions to `ResolvedReleased` or `ResolvedRefunded`.

EF Core mapping:

```
csharp
```

```
entity.HasIndex(e => e.BookingId).IsUnique();
entity.HasOne(e => e.DispatchedEmployee)
    .WithMany().HasForeignKey(e => e.DispatchedEmployeeId);
entity.HasOne(e => e.DispatchedByAdmin)
    .WithMany().HasForeignKey(e => e.DispatchedByAdminId);
entity.HasOne(e => e.DecidedByAdmin)
    .WithMany().HasForeignKey(e => e.DecidedByAdminId).IsRequired(false);
entity.HasOne(e => e.LinkedEvidenceForm)
    .WithMany().HasForeignKey(e => e.LinkedEvidenceFormId);
```

6.8 FieldVerificationPhotos — NEW

Purpose: Photos taken by the ShareGear employee at the job site. These are the counterpart to JobCompletionEvidencePhotos. The employee is expected to photograph the same elements the provider photographed — the work area, the completed task, and relevant site details — so the AI can make a meaningful visual comparison.

The admin dashboard presents the two photo sets side-by-side, grouped by DisplayOrder, when reviewing a FieldVerificationForm. The AiSimilarityScore on the parent form gives the composite assessment; individual photo comparisons are not stored separately.

Column	Type	Nullable	Notes
Id	uniqueidentifier	NO	PK
FieldVerificationFormId	uniqueidentifier	NO	FK → FieldVerificationForms
PhotoUrl	nvarchar(500)	NO	Azure Blob Storage URL
Caption	nvarchar(200)	YES	Employee label (e.g., "Same rooftop area — no equipment present")
DisplayOrder	int	NO	Default 0; should mirror provider photo ordering for easy comparison
CreatedAt	datetime2	NO	UTC

Minimum photos required: 1. Maximum: 15 (employees may need to document multiple angles). Enforced in FieldVerificationService before VisitStatus transitions to

Completed).

6.9 Bookings — Modified Columns Only

The following column is added. All other v1 columns are unchanged.

Column	Type	Nullable	Change
FieldVerificationDispatchedAt	datetime2	YES	NEW — set when FieldVerificationForm is created; used by admin SLA tracking

The Status column gains values 14 (PendingFieldVerification) and 15 (FieldVerificationComplete) per the updated BookingStatus enum in Section 3.1.

6.10 PlatformSettings — New Seeded Keys

Two new keys are added to the existing settings store. No schema change.

Key	Default Value	Description
FieldVerificationDispatchSlaHours	4	Hours within which an employee must be dispatched after a booking enters DISPUTED
FieldVerificationCompletionSlaHours	24	Hours within which the employee must submit the completed form after dispatch

7. Complete Workflow: Frozen Escrow → Field Verification → Resolution

This section documents the end-to-end flow that the three new evidence tables support together.

① Customer disputes completion

- `Bookings.Status = DISPUTED`
- `EscrowRecords.Status = FROZEN`
- Tickets (Category=CompletionDispute) AUTO-CREATED + linked to BookingId
- TicketMessage (SystemMessage): "Booking disputed. Escrow frozen."
- Notifications → Customer, Provider, Admin

② Admin reviews the ticket thread

- Reads JobCompletionEvidenceForms + JobCompletionEvidencePhotos
- Reads TicketMessages (customer's dispute description)
- Decision: dispatch field employee?
 - YES → proceeds to ③
 - NO → Admin resolves directly:
 - Tickets.ResolutionType set
 - EscrowService.Release() or .Refund()
 - Bookings.Status = RESOLVED_RELEASED / RESOLVED_REFUNDED
 - Ticket.Status = RESOLVED

③ Admin dispatches employee

- FieldVerificationForms created:
 - DispatchedEmployeeId, DispatchedByAdminId,
 - LinkedEvidenceFormId = JobCompletionEvidenceForms.Id,
 - DispatchInstructions, VisitStatus = DISPATCHED
- `Bookings.Status = PENDING_FIELD_VERIFICATION`
- `Bookings.FieldVerificationDispatchedAt = NOW()`
- AuditLogs record written
- Notification → Employee (FieldVerificationDispatched)
- TicketMessage (SystemMessage): "Field verification employee dispatched."
- Tickets.Status = PENDING_FIELD_VERIFICATION

④ Employee arrives on site

- Employee updates FieldVerificationForms.VisitStatus = ON_SITE
- FieldVerificationForms.VisitedAt = NOW()
- Notification → Admin (employee on-site)

⑤ Employee completes visit & submits form

- Employee uploads FieldVerificationPhotos (min 1, max 15)
- Employee fills EmployeeNotes
- Employee sets FieldVerificationOutcome (JobConfirmed / Not / Partial / Inconclusive)
- FieldVerificationForms.VisitStatus = COMPLETED
- FieldVerificationForms.FormSubmittedAt = NOW()

```

⑥ AI similarity scoring (async background service)
  → Semantic Kernel agent fetches JobCompletionEvidencePhotos URLs
  → Fetches FieldVerificationPhotos URLs
  → Azure OpenAI vision model compares both sets
  → Stores AiSimilarityScore (0.0000-1.0000) + AiSimilarityNotes
  → Bookings.Status = FIELD_VERIFICATION_COMPLETE
  → Notification → Admin (field verification complete; score: X)
  → TicketMessage (SystemMessage): "Field verification complete. AI
similarity: X%"

⑦ Admin makes final decision
  → Admin reviews: AiSimilarityScore, AiSimilarityNotes, EmployeeNotes,
    FieldVerificationOutcome, side-by-side photos in dashboard
  → Admin sets FieldVerificationForms.AdminDecision:
    RELEASE_TO_PROVIDER → EscrowService.Release()
                          → Bookings.Status = RESOLVED_RELEASED
    REFUND_TO_CUSTOMER  → EscrowService.Refund()
                          → Bookings.Status = RESOLVED_REFUNDED
    PARTIAL_SETTLEMENT  → EscrowService.PartialSettle(customerAmt,
providerAmt)
                          → Bookings.Status = RESOLVED_RELEASED (partial)
  → FieldVerificationForms.DecidedByAdminId + DecidedAt set
  → AuditLogs record written
  → Ticket.AdminResolution set; Ticket.Status = RESOLVED
  → TicketMessage (SystemMessage): "Dispute resolved: [decision summary]"
  → Notifications → Customer + Provider (resolution + amounts)
  → EmployeeProfiles.TotalVerificationsCompleted += 1

```

8. Normalization Review — v2 Additions

All v1 normalization conclusions remain valid. v2 additions evaluated below.

Tickets — fully normalized. **AiSummary**, **AiIdentifiedIssue**, and **AiClaimedImpact** are not transitive dependencies; they are AI-computed attributes of the ticket entity itself, stored inline for the same reason as the risk flags in **ServiceListings**. No normalization concern.

TicketMessages — fully normalized. **MessageType** is an attribute of the message, not a dependency on another column. **IsInternal** is a direct property of the message.

TicketAttachments — fully normalized. **FileName** is stored alongside **FileUrl** for display purposes — it is an independent attribute of the upload, not derivable from **FileUrl** (blob URLs are opaque GUIDs).

`JobCompletionEvidenceForms` — fully normalized. `ReviewedByAdminId` and `AdminReviewNote` are populated only during dispute resolution and are direct attributes of the form's review lifecycle, not transitive.

`FieldVerificationForms` — one deliberate justified denormalization: `LinkedEvidenceFormId` FK could theoretically be derived by querying `JobCompletionEvidenceForms WHERE BookingId = this.BookingId`. Storing it as a direct FK is intentional — it makes the relationship explicit in the schema, enables a direct join in the AI comparison service without an intermediate lookup, and protects against the edge case where a booking's evidence form could theoretically be resubmitted (resubmission replaces the record, but the FV form should compare against the specific form version that existed at dispatch time).

`AiSimilarityScore` and `AiSimilarityNotes` on `FieldVerificationForms` — stored inline. The AI output is an attribute of the field verification assessment, not a first-class entity. No separate AI results table is warranted.

Verdict: All 8 new tables are 3NF-compliant with the documented, justified exceptions. No refactoring required.

9. Coverage Validation — v2

Requirement	Table(s)	Status
Replace Complaints with full ticket system	<code>Tickets</code> , <code>TicketMessages</code> , <code>TicketAttachments</code>	✓ Covered
Ticket follow-up threading	<code>TicketMessages.TicketId</code>	✓ Covered
Internal agent notes (invisible to users)	<code>TicketMessages.IsInternal</code>	✓ Covered
Ticket assignment to employees	<code>Tickets.AssignedToUserId</code> → Employee	✓ Covered
Human-readable ticket reference number	<code>Tickets.TicketNumber</code>	✓ Covered

Requirement	Table(s)	Status
Ticket linked to booking or order	<code>Tickets.BookingId</code> , <code>Tickets.MarketplaceOrderId</code>	✓ Covered
Auto-create ticket on booking dispute	Service layer + <code>Tickets.Category=CompletionDispute</code>	✓ Covered
Provider completion evidence form	<code>JobCompletionEvidenceForms</code>	✓ Covered
Provider completion photos	<code>JobCompletionEvidencePhotos</code>	✓ Covered
Form required at job-complete step	<code>BookingService</code> guard + <code>EvidenceFormStatus=Submitted</code>	✓ Covered
Employee dispatch for frozen escrow	<code>FieldVerificationForms.DispatchedEmployeeId</code>	✓ Covered
Employee on-site photo capture	<code>FieldVerificationPhotos</code>	✓ Covered
AI photo comparison (provider vs employee)	<code>FieldVerificationForms.AiSimilarityScore</code> , <code>AiSimilarityNotes</code>	✓ Covered
Admin sees both photo sets side-by-side	<code>LinkedEvidenceFormId</code> FK enables joined query	✓ Covered
Admin final escrow decision from FV	<code>FieldVerificationForms.AdminDecision</code> → <code>EscrowService</code>	✓ Covered
Employee role and profile	<code>Users.Role=3</code> , <code>EmployeeProfiles</code>	✓ Covered
Field ops SLA tracking	<code>FieldVerificationForms.DispatchedAt</code> , <code>FormSubmittedAt</code> + <code>PlatformSettings</code> keys	✓ Covered
Dispatch instructions to employee	<code>FieldVerificationForms.DispatchInstructions</code>	✓ Covered

Requirement	Table(s)	Status
Employee outcome assessment	<code>FieldVerificationForms.FieldVerificationOutcome</code>	✓ Covered
Employee performance tracking	<code>EmployeeProfiles.TotalVerificationsCompleted</code>	✓ Covered
All admin actions on new entities audited	<code>AuditLogs</code> — no schema change needed	✓ Covered
Notifications for new workflow events	<code>Notifications.EventType</code> — new event type strings, no schema change	✓ Covered

No uncovered requirements identified.

10. Updated Implementation Order

Sprints 1-5 from v1 are unchanged. Sprints 6-8 are updated.

Sprint 6 — Ticket System & Post-Transaction (replaces v1 Sprint 6)

Tables: `Tickets`, `TicketMessages`, `TicketAttachments`, `Reviews`

Implement ticket creation, agent assignment, threaded messaging, internal notes, AI summarisation hook on ticket open, `TicketNumber` generation (format: `TKT-{YYYYMMDD}-{sequence}`), auto-create ticket on booking dispute, filtered unique index on `BookingId`.

Sprint 7 — Job Completion Evidence (new)

Tables: `JobCompletionEvidenceForms`, `JobCompletionEvidencePhotos`

Implement the provider-facing completion form UI and API. Wire into `BookingService.MarkJobComplete()` so that the form must be submitted before the booking status transitions. Implement the 30-minute reminder background job.

Sprint 8 — Field Verification & Employee Workflows (new)

Tables: `FieldVerificationForms`, `FieldVerificationPhotos`, `EmployeeProfiles`

Implement employee profile management. Implement admin dispatch endpoint (creates `FieldVerificationForm`, transitions booking to `PendingFieldVerification`). Implement

employee-facing form submission API. Wire the async AI similarity scoring background service. Implement admin decision endpoint that invokes `EscrowService`. Update `TicketService` to mirror booking status changes as system messages.

Sprint 9 — Operations, Admin & Reputation (v1 Sprint 7, renumbered)

Tables: `ProviderIncidents`, `AuditLogs`

Unchanged from v1 Sprint 7.

Sprint 10 — Polish & Reliability (v1 Sprint 8, renumbered)

All v1 Sprint 8 tasks plus:

- New indexes from v2 appendix below
- Field verification SLA monitoring in admin dashboard
- Side-by-side photo comparison view in admin dispute resolution panel

Appendix A — New & Updated SQL Server Indexes (v2 additions)

sql

```

-- Ticket lookup by booking (most common dispute query)
CREATE UNIQUE INDEX UX_Tickets_BookingId_Active
ON Tickets (BookingId)
WHERE BookingId IS NOT NULL AND Status <> 6; -- 6 = Closed

-- Ticket queue for agent (assigned + open)
CREATE INDEX IX_Tickets_AssignedToUserId_Status
ON Tickets (AssignedToUserId, Status, Priority DESC, CreatedAt);

-- Ticket message thread retrieval
CREATE INDEX IX_TicketMessages_TicketId_CreatedAt
ON TicketMessages (TicketId, CreatedAt);

-- Internal notes filter (agent dashboard)
CREATE INDEX IX_TicketMessages_TicketId_IsInternal
ON TicketMessages (TicketId, IsInternal, CreatedAt);

-- Evidence photo gallery (ordered)
CREATE INDEX IX_JobCompletionEvidencePhotos_FormId_Order
ON JobCompletionEvidencePhotos (FormId, DisplayOrder);

-- One evidence form per booking
CREATE UNIQUE INDEX UX_JobCompletionEvidenceForms_BookingId
ON JobCompletionEvidenceForms (BookingId);

-- One field verification form per booking
CREATE UNIQUE INDEX UX_FieldVerificationForms_BookingId
ON FieldVerificationForms (BookingId);

-- Employee dispatch queue (available employees by governorate)
CREATE INDEX IX_EmployeeProfiles_Dispatch
ON EmployeeProfiles (IsAvailableForDispatch, AssignedGovernorate);

-- Field verification photo gallery (ordered)
CREATE INDEX IX_FieldVerificationPhotos_FormId_Order
ON FieldVerificationPhotos (FieldVerificationFormId, DisplayOrder);

-- Admin: pending field verification decisions
CREATE INDEX IX_FieldVerificationForms_AdminDecision_Status
ON FieldVerificationForms (AdminDecision, VisitStatus, DispatchedAt);

-- Ticket attachments for a message

```

```
CREATE INDEX IX_TicketAttachments_TicketMessageId
ON TicketAttachments (TicketMessageId);
```

Appendix B — Updated EF Core DbContext (v2)

```
csharp
```

```

public class ShareGearDbContext : IdentityDbContext<User, IdentityRole<Guid>, G
{
    // — v1 unchanged —————
    public DbSet<RefreshToken> RefreshTokens => Set<RefreshToken>();
    public DbSet<Address> Addresses => Set<Address>();
    public DbSet<ProviderProfile> ProviderProfiles => Set<ProviderProfile>();
    public DbSet<Operator> Operators => Set<Operator>();
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<ServiceListing> ServiceListings => Set<ServiceListing>();
    public DbSet<ServiceListingPhoto> ServiceListingPhotos => Set<ServiceListin
    public DbSet<BlackoutDate> BlackoutDates => Set<BlackoutDate>();
    public DbSet<Document> Documents => Set<Document>();
    public DbSet<Booking> Bookings => Set<Booking>();
    public DbSet<OperatorAssignment> OperatorAssignments => Set<OperatorAssignm
    public DbSet<EscrowRecord> EscrowRecords => Set<EscrowRecord>();
    public DbSet<MarketplaceListing> MarketplaceListings => Set<MarketplaceList
    public DbSet<MarketplaceListingPhoto> MarketplaceListingPhotos => Set<Marke
    public DbSet<MarketplaceOrder> MarketplaceOrders => Set<MarketplaceOrder>()
    public DbSet<Offer> Offers => Set<Offer>();
    public DbSet<Review> Reviews => Set<Review>();
    public DbSet<Conversation> Conversations => Set<Conversation>();
    public DbSet<Message> Messages => Set<Message>();
    public DbSet<Notification> Notifications => Set<Notification>();
    public DbSet<ProviderIncident> ProviderIncidents => Set<ProviderIncident>()
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();
    public DbSet<PlatformSetting> PlatformSettings => Set<PlatformSetting>();

    // — v2 new —————
    public DbSet<EmployeeProfile> EmployeeProfiles => Set<EmployeeProfile>();
    public DbSet<Ticket> Tickets => Set<Ticket>();
    public DbSet<TicketMessage> TicketMessages => Set<TicketMessage>();
    public DbSet<TicketAttachment> TicketAttachments => Set<TicketAttachment>()
    public DbSet<JobCompletionEvidenceForm> JobCompletionEvidenceForms
        => Set<JobCompletionEvidenceForm>();
    public DbSet<JobCompletionEvidencePhoto> JobCompletionEvidencePhotos
        => Set<JobCompletionEvidencePhoto>();
    public DbSet<FieldVerificationForm> FieldVerificationForms
        => Set<FieldVerificationForm>();
    public DbSet<FieldVerificationPhoto> FieldVerificationPhotos
        => Set<FieldVerificationPhoto>();
}

```

Signed off — Principal Solution Architect & Senior Database Architect. Version 2.0. 33 tables. All new requirements covered. No uncovered workflow gaps. Ready for SQL Server migration generation and EF Core entity scaffolding.